

10.9 Lab: Deep Learning

Trong section này, ta demonstrate cách fit các examples đã thảo luận trong text. Ta dùng Python package `torch`, cùng với `pytorch_lightning`, package cung cấp các utilities để đơn giản hóa việc fitting và evaluating models. Code này có thể chạy rất nhanh trên một số special processors, chẳng hạn chip M1 mới của Apple. Package có structure tốt, flexible và sẽ khá quen thuộc với Python users. Một companion hữu ích là site pytorch.org/tutorials. Phần lớn code của ta được adapted từ đó, cũng như từ documentation của `pytorch_lightning`.²³

Ta bắt đầu với một số standard imports đã gặp trước đây.

In [1]:

```
import numpy as np, pandas as pd
from matplotlib.pyplot import subplots
from sklearn.linear_model import \
    (LinearRegression,
     LogisticRegression,
     Lasso)
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import KFold
from sklearn.pipeline import Pipeline
from ISLP import load_data
from ISLP.models import ModelSpec as MS
from sklearn.model_selection import \
    (train_test_split,
     GridSearchCV)
```

Torch-Specific Imports

Có một số imports dành cho `torch`. (Chúng không đi kèm với ISLP, nên phải được install riêng.) Trước hết ta import main library và các essential tools dùng để specify sequentially-structured networks.

In [2]:

```
import torch
from torch import nn
from torch.optim import RMSprop
from torch.utils.data import TensorDataset
```

Có một số helper packages khác cho `torch`. Chẳng hạn, package `torchmetrics` có utilities để compute nhiều metrics nhằm evaluate performance khi fitting một model. Package `torchinfo` cung cấp summary hữu ích về các layers của model. Ta dùng function `read_image()` khi loading test images trong Section 10.9.4.

`torchmetrics;`
`torchinfo;`
`read_image()`In [3]:

```
from torchmetrics import (MeanAbsoluteError,
                          R2Score)
from torchinfo import summary
from torchvision.io import read_image
```

Package `pytorch_lightning` là một interface ở somewhat higher level cho `torch`, giúp simplify việc specification và fitting models bằng cách giảm lượng boilerplate code cần thiết so với dùng `torch` trực tiếp.

²³Các URL chính xác tại thời điểm viết là <https://pytorch.org/tutorials/beginner/basics/intro.html> và <https://pytorch-lightning.readthedocs.io/en/latest/>.

```
In [4]: from pytorch_lightning import Trainer
        from pytorch_lightning.loggers import CSVLogger
```

Để reproduce results, ta dùng `seed_everything()`. Ta cũng instruct `torch` dùng deterministic algorithms khi có thể.

seed_everything()

```
In [5]: from pytorch_lightning.utilities.seed import seed_everything
        seed_everything(0, workers=True)
        torch.use_deterministic_algorithms(True, warn_only=True)
```

Ta sẽ dùng một số datasets được ship cùng `torchvision` cho examples: một pretrained network cho image classification, cùng một số transforms dùng cho preprocessing.

torchvision

```
In [6]: from torchvision.datasets import MNIST, CIFAR100
        from torchvision.models import (resnet50,
                                       ResNet50_Weights)
        from torchvision.transforms import (Resize,
                                           Normalize,
                                           CenterCrop,
                                           ToTensor)
```

Ta đã cung cấp một vài utilities trong ISLP dành riêng cho lab này. `SimpleDataModule` và `SimpleModule` là các simple versions của objects dùng trong `pytorch_lightning`, high-level module để fit `torch` models. Dù module này cho phép những advanced uses như computing trên graphical processing units (GPUs) và parallel data processing, ta sẽ không tập trung vào chúng trong lab này. `ErrorTracker` xử lý collections của targets và predictions trên từng mini-batch trong validation hoặc test stage, cho phép compute metric trên toàn bộ validation hoặc test data set.

```
In [7]: from ISLP.torch import (SimpleDataModule,
                               SimpleModule,
                               ErrorTracker,
                               rec_num_workers)
```

Ngoài ra ta còn include một số helper functions để load IMDb database, cùng một lookup map integers tới particular keys trong database. Ta đã include một slightly modified copy của preprocessed IMDb data từ `keras`, một package riêng để fitting deep learning models. Điều này tiết kiệm đáng kể preprocessing và cho phép ta tập trung vào specification và fitting models.

keras

```
In [8]: from ISLP.torch.imdb import (load_lookup,
                                       load_tensor,
                                       load_sparse,
                                       load_sequential)
```

Cuối cùng, ta giới thiệu một số utility imports không liên quan trực tiếp tới `torch`. Function `glob()` từ module `glob` dùng để tìm tất cả files matching wildcard characters; ta sẽ dùng nó trong example apply model ResNet50 lên một số images của chính mình. Module `json` sẽ được dùng để load một JSON file nhằm lookup classes để identify labels của pictures trong example ResNet50.

glob(); json

In [9]:

```
from glob import glob
import json
```

10.9.1 Single Layer Network on Hitters Data

Ta bắt đầu bằng cách fit các models trong Section 10.6 trên `Hitters` data.

In [10]:

```
Hitters = load_data('Hitters').dropna()
n = Hitters.shape[0]
```

Ta sẽ fit hai linear models (least squares và lasso) và compare performance của chúng với neural network. Trong comparison này, ta dùng mean absolute error trên một validation dataset,

$$\text{MAE}(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

Ta setup model matrix và response.

In [11]:

```
model = MS(Hitters.columns.drop('Salary'), intercept=False)
X = model.fit_transform(Hitters).to_numpy()
Y = Hitters['Salary'].to_numpy()
```

Method `to_numpy()` ở trên convert pandas data frames hoặc series thành numpy arrays. Ta làm vậy vì sẽ cần dùng `sklearn` để fit lasso model, và nó yêu cầu conversion này. Ta cũng dùng một linear regression method từ `sklearn`, thay vì method trong Chapter 3 từ `statsmodels`, để facilitate comparisons.

`to_numpy()`

Bây giờ ta split data thành test và training, fixing random state mà `sklearn` dùng để thực hiện split.

In [12]:

```
(X_train,
 X_test,
 Y_train,
 Y_test) = train_test_split(X,
                             Y,
                             test_size=1/3,
                             random_state=1)
```

Linear Models

Ta fit linear model và evaluate test error trực tiếp.

In [13]:

```
hit_lm = LinearRegression().fit(X_train, Y_train)
Yhat_test = hit_lm.predict(X_test)
np.abs(Yhat_test - Y_test).mean()
```

Out[13]:

259.7153

Tiếp theo ta fit lasso bằng `sklearn`. Ở đây ta dùng mean absolute error để select và evaluate model, thay vì mean squared error. Specialized solver dùng trong Section 6.5.2 chỉ dùng mean squared error. Vì vậy ở đây, với thêm một chút work, ta tạo cross-validation grid và thực hiện cross-validation trực tiếp.

Ta encode một pipeline với hai steps: trước hết normalize features bằng transform `StandardScaler()`, sau đó fit lasso mà không normalize thêm.

In [14]:

```
scaler = StandardScaler(with_mean=True, with_std=True)
lasso = Lasso(warm_start=True, max_iter=30000)
standard_lasso = Pipeline(steps=[('scaler', scaler),
                                  ('lasso', lasso)])
```

Ta cần tạo một grid các values của λ . Theo common practice, ta chọn grid gồm 100 values của λ , uniform trên log scale từ `lam_max` xuống $0.01 \times \text{lam_max}$. Ở đây `lam_max` là smallest value của λ với all-zero solution. Value này bằng largest absolute inner-product giữa bất kỳ predictor nào và centered response.²⁴

In [15]:

```
X_s = scaler.fit_transform(X_train)
n = X_s.shape[0]
lam_max = np.fabs(X_s.T.dot(Y_train - Y_train.mean())).max() / n
param_grid = {'alpha': np.exp(np.linspace(0, np.log(0.01), 100))
               * lam_max}
```

Lưu ý ta phải transform data trước vì scale của variables ảnh hưởng tới choice của λ . Bây giờ ta perform cross-validation bằng sequence các λ values này.

In [16]:

```
cv = KFold(10,
           shuffle=True,
           random_state=1)
grid = GridSearchCV(lasso,
                    param_grid,
                    cv=cv,
                    scoring='neg_mean_absolute_error')
grid.fit(X_train, Y_train);
```

Ta extract lasso model có best cross-validated mean absolute error, rồi evaluate performance của nó trên `X_test` và `Y_test`, vốn không được dùng trong cross-validation.

In [17]:

```
trained_lasso = grid.best_estimator_
Yhat_test = trained_lasso.predict(X_test)
np.fabs(Yhat_test - Y_test).mean()
```

Out[17]: 257.2382

Kết quả này tương tự những gì ta nhận được với linear model fit bằng least squares. Tuy nhiên results có thể vary nhiều với different train/test splits; reader được khuyến khích thử different seed trong code block 12 và rerun các code blocks tiếp theo tới point này.

Specifying a Network: Classes and Inheritance

Để fit neural network, trước hết ta setup một model structure mô tả network. Việc này yêu cầu define các new classes specific cho model muốn fit. Thông thường điều này được thực hiện trong `pytorch` bằng cách subclass một generic representation của network; đây cũng là approach ta dùng. Dù example này simple, ta sẽ đi qua các steps khá chi tiết vì chúng sẽ hữu

²⁴Derivation của result này nằm ngoài scope của book.

ích cho các examples phức tạp hơn phía sau.

In [18]:

```
class HittersModel(nn.Module):

    def __init__(self, input_size):
        super(HittersModel, self).__init__()
        self.flatten = nn.Flatten()
        self.sequential = nn.Sequential(
            nn.Linear(input_size, 50),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(50, 1))

    def forward(self, x):
        x = self.flatten(x)
        return torch.flatten(self.sequential(x))
```

Statement `class` identifies code chunk như declaration cho class `HittersModel`, class này inherits từ base class `nn.Module`. Base class này xuất hiện khắp `torch` và represent mappings trong neural networks.

Indented dưới statement `class` là methods của class: ở đây là `__init__` và `forward`. Method `__init__` được gọi khi một instance của class được create như trong cell dưới. Trong methods, `self` luôn refer tới một instance của class. Trong `__init__`, ta attach hai objects vào `self` như attributes: `flatten` và `sequential`. Chúng được dùng trong `forward` để describe map mà module implements.

Có thêm một line trong method `__init__`: call tới `super()`. Function này cho phép subclasses (ví dụ `HittersModel`) access methods của class mà chúng inherit. Chẳng hạn class `nn.Module` có method `__init__` riêng, khác với method `HittersModel.__init__()` ta vừa viết. Dùng `super()` cho phép call method của base class. Với `torch` models, ta luôn thực hiện call `super()` này vì nó cần thiết để model được `torch` interpret đúng.

Object `nn.Module` có nhiều methods hơn chỉ `__init__` và `forward`. Các methods này trực tiếp accessible cho `HittersModel` instances nhờ inheritance. Một method sẽ gặp ngay sau đây là `eval()`, dùng để disable dropout khi ta muốn evaluate model trên test data.

In [19]:

```
hit_model = HittersModel(X.shape[1])
```

Object `self.sequential` là composition của bốn maps. Map đầu tiên map 19 features của `Hitters` sang 50 dimensions, introducing $50 \times 19 + 50$ parameters cho weights và intercept của map (thường gọi là bias). Layer này sau đó được map tới một ReLU layer, tiếp theo là 40% dropout layer, và cuối cùng là một linear map xuống 1 dimension, lại có bias. Tổng số trainable parameters vì vậy là $50 \times 19 + 50 + 50 + 1 = 1051$.

Package `torchinfo` cung cấp function `summary()` summarize information này một cách gọn gàng. Ta specify size của input và xem size của mỗi tensor khi đi qua các layers của network.

In [20]:

```
summary(hit_model,
        input_size=X_train.shape,
        col_names=['input_size',
                   'output_size',
                   'num_params'])
```

Out[20]:

```

=====
Layer (type:depth-idx)      Input Shape    Output Shape   Param #
=====
HittersModel                [175, 19]      [175]           --
  Flatten: 1-1               [175, 19]      [175, 19]       --
  Sequential: 1-2            [175, 19]      [175, 1]         --
    Linear: 2-1               [175, 19]      [175, 50]        1,000
    ReLU: 2-2                 [175, 50]      [175, 50]         --
    Dropout: 2-3              [175, 50]      [175, 50]         --
    Linear: 2-4                [175, 50]      [175, 1]          51
=====
Total params: 1,051
Trainable params: 1,051

```

Ta đã truncate nhẹ phần cuối của output ở đây và ở các uses sau. Bây giờ ta cần transform training data sang form accessible cho **torch**. Basic datatype trong **torch** là tensor, rất giống **ndarray** ở các chapters đầu. Cũng lưu ý **torch** thường làm việc với 32-bit (single precision) thay vì 64-bit (double precision) floating point numbers. Vì vậy ta convert data sang **np.float32** trước khi forming tensor. Các *X* và *Y* tensors sau đó được arranged vào một **Dataset** recognized bởi **torch** bằng **TensorDataset()**.

Dataset;
TensorDataset()

In [21]:

```

X_train_t = torch.tensor(X_train.astype(np.float32))
Y_train_t = torch.tensor(Y_train.astype(np.float32))
hit_train = TensorDataset(X_train_t, Y_train_t)

```

Ta làm tương tự cho test data.

In [22]:

```

X_test_t = torch.tensor(X_test.astype(np.float32))
Y_test_t = torch.tensor(Y_test.astype(np.float32))
hit_test = TensorDataset(X_test_t, Y_test_t)

```

Cuối cùng dataset này được pass vào một **DataLoader()** mà sau cùng sẽ pass data vào network. Dù có vẻ khá nhiều overhead, structure này hữu ích cho tasks phức tạp hơn nơi data có thể nằm trên different machines hoặc phải pass tới GPU. Ta cung cấp helper function **SimpleDataModule()** trong ISLP để task này dễ hơn cho standard usage. Một argument của nó là **num_workers**, chỉ số processes sẽ dùng để loading data. Với small data như **Hitters**, điều này ít ảnh hưởng, nhưng có advantage cho examples **MNIST** và **CIFAR100** bên dưới. Package **torch** sẽ inspect process đang chạy và determine maximum number of workers.²⁵ Ta đã include function **rec_num_workers()** để compute value này nhằm biết số workers hợp lý (ở đây maximum là 16).

In [23]:

```

max_num_workers = rec_num_workers()

```

General training setup trong **pytorch_lightning** gồm training, validation và test data. Mỗi loại được represented bởi different data loaders. Trong mỗi epoch, ta run training step để learn model và validation step để track error. Test data thường được dùng ở cuối training để evaluate model.

Trong case này, vì chỉ split thành test và training, ta dùng test data làm validation data với argument **validation=hit_test**. Argument **validation** có thể là float giữa 0 và 1, integer, hoặc **Dataset**. Nếu là float (tương ứng integer), nó được interpret như percentage (tương ứng

²⁵Điều này phụ thuộc vào computing hardware và number of cores khả dụng.

number) của training observations dùng cho validation. Nếu là `Dataset`, nó được pass trực tiếp tới data loader.

In [24]:

```
hit_dm = SimpleDataModule(hit_train,
                          hit_test,
                          batch_size=32,
                          num_workers=min(4, max_num_workers),
                          validation=hit_test)
```

Tiếp theo ta phải cung cấp một `pytorch_lightning` module control các steps được thực hiện trong training process. Ta cung cấp methods cho `SimpleModule()` đơn giản record value của loss function và bất kỳ additional metrics nào ở cuối mỗi epoch. Các operations này được controlled bởi methods `SimpleModule.[training/test/validation]_step()`, dù ta không modify chúng trong examples.

In [25]:

```
hit_module = SimpleModule.regression(hit_model,
                                     metrics={'mae': MeanAbsoluteError()})
```

Bằng cách dùng method `SimpleModule.regression()`, ta indicate sẽ dùng squared-error loss như trong (10.23). Ta cũng yêu cầu track mean absolute error trong metrics được logged.

Ta log results qua `CSVLogger()`, trong case này lưu results vào CSV file trong directory `logs/hitters`. Sau khi fitting hoàn tất, điều này cho phép load results thành `pd.DataFrame()` và visualize ở dưới. Có nhiều cách log results trong `pytorch_lightning`, nhưng ta không cover chi tiết.

In [26]:

```
hit_logger = CSVLogger('logs', name='hitters')
```

Cuối cùng ta sẵn sàng train model và log results. Ta dùng object `Trainer()` từ `pytorch_lightning`. Argument `datamodule=hit_dm` tells trainer cách produce training/validation/test logs, còn first argument `hit_module` specifies network architecture cũng như training/validation/test steps. Argument `callbacks` cho phép nhiều tasks được thực hiện ở various points khi training model. Ở đây callback `ErrorTracker()` cho phép compute validation error trong training và cuối cùng test error. Ta fit model trong 50 epochs.

In [27]:

```
hit_trainer = Trainer(deterministic=True,
                     max_epochs=50,
                     log_every_n_steps=5,
                     logger=hit_logger,
                     callbacks=[ErrorTracker()])
hit_trainer.fit(hit_module, datamodule=hit_dm)
```

Ở mỗi step của SGD, algorithm randomly selects 32 training observations để compute gradient. Nhớ từ Section 10.7 rằng một epoch là number of SGD steps cần để process n observations. Vì training set có $n = 175$, và ta specify `batch_size` là 32 khi construct `hit_dm`, một epoch là $175/32 = 5.5$ SGD steps.

Sau khi fit model, ta evaluate performance trên test data bằng method `test()` của trainer.

In [28]: `hit_trainer.test(hit_module, datamodule=hit_dm)`

Out[28]: `[{'test_loss': 104098.5469, 'test_mae': 229.5012}]`

Results của fit đã được logged vào CSV file. Ta có thể tìm results specific cho run này trong attribute `experiment.metrics_file_path` của logger. Mỗi lần model được fit, logger output results vào một new subdirectory trong directory `logs/hitters`.

Bây giờ ta tạo plot của MAE (mean absolute error) như function của number of epochs. Trước hết retrieve logged summaries.

```
hit_results = pd.read_csv(hit_logger.experiment.metrics_file_path)
```

Vì sẽ produce các plots tương tự ở later examples, ta viết một simple generic function để produce plot này.

```
In [29]: def summary_plot(results,
                ax,
                col='loss',
                valid_legend='Validation',
                training_legend='Training',
                ylabel='Loss',
                fontsize=20):
    for (column,
         color,
         label) in zip([f'train_{col}_epoch',
                        f'valid_{col}'],
                       ['black',
                        'red'],
                       [training_legend,
                        valid_legend]):
        results.plot(x='epoch',
                     y=column,
                     label=label,
                     marker='o',
                     color=color,
                     ax=ax)
    ax.set_xlabel('Epoch')
    ax.set_ylabel(ylabel)
    return ax
```

Ta setup axes rồi dùng function vừa viết để produce MAE plot.

```
In [30]: fig, ax = subplots(1, 1, figsize=(6, 6))
ax = summary_plot(hit_results,
                ax,
                col='mae',
                ylabel='MAE',
                valid_legend='Validation (=Test)')
ax.set_ylim([0, 400])
ax.set_xticks(np.linspace(0, 50, 11).astype(int));
```

Ta có thể predict trực tiếp từ final model và evaluate performance trên test data. Trước prediction, ta call method `eval()` của `hit_model`. Điều này tells `torch` effectively consider model đã fitted để có thể predict trên new data. Với model này, thay đổi lớn nhất là dropout layers được turn off, tức không có weights bị randomly dropped khi predicting new data.

```
In [31]: hit_model.eval()
         preds = hit_module(X_test_t)
         torch.abs(Y_test_t - preds).mean()
```

```
Out[31]: tensor(229.5012, grad_fn=<MeanBackward0>)
```

Cleanup

Khi setup data module, ta đã initiate một số worker processes sẽ tiếp tục running. Ta delete mọi references tới `torch` objects để đảm bảo các processes này bị killed.

```
In [32]: del(Hitters,
            hit_model, hit_dm,
            hit_logger,
            hit_test, hit_train,
            X, Y,
            X_test, X_train,
            Y_test, Y_train,
            X_test_t, Y_test_t,
            hit_trainer, hit_module)
```

10.9.2 Multilayer Network on the MNIST Digit Data

Package `torchvision` đi kèm một số example datasets, gồm MNIST digit data. Step đầu là retrieve training và test data sets; function `MNIST()` trong `torchvision.datasets` được cung cấp cho purpose này. Data sẽ được downloaded ở lần đầu function được execute và stored trong directory `data/MNIST`.

`MNIST()`

```
In [33]: (mnist_train,
          mnist_test) = [MNIST(root='data',
                               train=train,
                               download=True,
                               transform=ToTensor())
                        for train in [True, False]]
          mnist_train
```

```
Out[33]: Dataset MNIST
         Number of datapoints: 60000
         Root location: data
         Split: Train
         StandardTransform
         Transform: ToTensor()
```

Có 60,000 images trong training data và 10,000 trong test data. Images là 28×28 và stored như matrix of pixels. Ta cần transform mỗi image thành vector.

Neural networks somewhat sensitive với scale của inputs, tương tự ridge và lasso regularization bị affected bởi scaling. Ở đây inputs là eight-bit grayscale values từ 0 đến 255, nên ta rescale sang unit interval.²⁶ Transformation này, cùng một số reordering của axes, được thực hiện bởi transform `ToTensor()` từ package `torchvision.transforms`.

²⁶Eight bits nghĩa là $2^8 = 256$. Vì convention bắt đầu từ 0, possible values nằm từ 0 đến 255.

Như trong `Hitters` example, ta form data module từ training và test datasets, setting aside 20% training images cho validation.

```
In [34]: mnist_dm = SimpleDataModule(mnist_train,
                                mnist_test,
                                validation=0.2,
                                num_workers=max_num_workers,
                                batch_size=256)
```

Ta xem data sẽ được fed vào network. Ta loop qua vài chunks đầu của test dataset, breaking sau 2 batches:

```
In [35]: for idx, (X_, Y_) in enumerate(mnist_dm.train_dataloader()):
          print('X: ', X_.shape)
          print('Y: ', Y_.shape)
          if idx >= 1:
              break
```

```
Out[35]: X: torch.Size([256, 1, 28, 28])
          Y: torch.Size([256])
          X: torch.Size([256, 1, 28, 28])
          Y: torch.Size([256])
```

Ta thấy X của mỗi batch gồm 256 images size $1 \times 28 \times 28$. Số 1 biểu thị single channel (greyscale). Với RGB images như CIFAR100 bên dưới, số 1 trong size sẽ được replace bằng 3 cho three RGB channels.

Bây giờ ta sẵn sàng specify neural network.

```
In [36]: class MNISTModel(nn.Module):
          def __init__(self):
              super(MNISTModel, self).__init__()
              self.layer1 = nn.Sequential(
                  nn.Flatten(),
                  nn.Linear(28*28, 256),
                  nn.ReLU(),
                  nn.Dropout(0.4))
              self.layer2 = nn.Sequential(
                  nn.Linear(256, 128),
                  nn.ReLU(),
                  nn.Dropout(0.3))
              self._forward = nn.Sequential(
                  self.layer1,
                  self.layer2,
                  nn.Linear(128, 10))
          def forward(self, x):
              return self._forward(x)
```

Trong first layer, mỗi $1 \times 28 \times 28$ image được flatten, rồi mapped tới 256 dimensions nơi ta apply ReLU activation với 40% dropout. Second layer map output của first layer xuống 128 dimensions, apply ReLU activation với 30% dropout. Cuối cùng 128 dimensions được mapped xuống 10, number of classes trong MNIST data.

```
In [37]: mnist_model = MNISTModel()
```

Ta check model produce output đúng expected size dựa trên existing batch $X_$ ở trên.

```
In [38]: mnist_model(X_).size()
```

```
Out[38]: torch.Size([256, 10])
```

Ta xem summary của model. Thay vì `input_size`, ta có thể pass một tensor có correct shape. Ở đây ta pass final batched `X_` ở trên.

```
In [39]: summary(mnist_model,
               input_data=X_,
               col_names=['input_size',
                          'output_size',
                          'num_params'])
```

```
Out[39]: =====
Layer (type:depth-idx) Input Shape          Output Shape Param #
=====
MNISTModel                [256, 1, 28, 28] [256, 10]      --
  Sequential: 1-1          [256, 1, 28, 28] [256, 10]      --
    Sequential: 2-1        [256, 1, 28, 28] [256, 256]     --
      Flatten: 3-1         [256, 1, 28, 28] [256, 784]     --
        Linear: 3-2        [256, 784]      [256, 256]    200,960
        ReLU: 3-3          [256, 256]      [256, 256]     --
        Dropout: 3-4       [256, 256]      [256, 256]     --
    Sequential: 2-2        [256, 256]      [256, 128]     --
      Linear: 3-5          [256, 256]      [256, 128]    32,896
      ReLU: 3-6            [256, 128]      [256, 128]     --
      Dropout: 3-7         [256, 128]      [256, 128]     --
      Linear: 2-3          [256, 128]      [256, 10]     1,290
=====
Total params: 235,146
Trainable params: 235,146
```

Sau khi setup cả model và data module, fitting model gần như identical với `Hitters` example. Khác với regression model, ở đây ta dùng method `SimpleModule.classification()`, dùng cross-entropy loss function thay vì mean squared error.

```
In [40]: mnist_module = SimpleModule.classification(mnist_model)
         mnist_logger = CSVLogger('logs', name='MNIST')
```

Bây giờ ta sẵn sàng. Final step là supply training data và fit model.

```
In [41]: mnist_trainer = Trainer(deterministic=True,
                                   max_epochs=30,
                                   logger=mnist_logger,
                                   callbacks=[ErrorTracker()])
         mnist_trainer.fit(mnist_module,
                           datamodule=mnist_dm)
```

Ta đã suppress output ở đây; output là progress report về fitting model, grouped by epoch. Điều này rất useful vì fitting trên large datasets có thể mất thời gian. Fitting model này mất 245 seconds trên MacBook Pro với Apple M1 Pro chip 10 cores và 16 GB RAM. Ta specify validation split 20%, nên training thực tế dùng 80% của 60,000 observations trong training set. Đây là alternative cho việc supply validation data trực tiếp như với `Hitters`. SGD dùng batches 256 observations khi computing gradient; tính ra một epoch tương ứng 188 gradient steps.

`SimpleModule.classification()` include accuracy metric mặc định. Có thể add other classification metrics từ `torchmetrics`. Ta dùng function `summary_plot()` để display accuracy qua epochs.

```
In [42]: mnist_results = pd.read_csv(mnist_logger.experiment.
                                     metrics_file_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(mnist_results,
              ax,
              col='accuracy',
              ylabel='Accuracy')
ax.set_ylim([0.5, 1])
ax.set_ylabel('Accuracy')
ax.set_xticks(np.linspace(0, 30, 7).astype(int));
```

Một lần nữa ta evaluate accuracy bằng method `test()` của trainer. Model này đạt 97% accuracy trên test data.

```
In [43]: mnist_trainer.test(mnist_module,
                             datamodule=mnist_dm)
```

```
Out[43]: [{'test_loss': 0.1471, 'test_accuracy': 0.9681}]
```

Table 10.1 cũng report error rates từ LDA (Chapter 4) và multiclass logistic regression. Với LDA, reader được refer tới Section 4.7.3. Dù có thể dùng function `LogisticRegression()` của `sklearn` để fit multiclass logistic regression, ở đây ta đã setup để fit model này bằng `torch`. Ta chỉ có input layer và output layer, bỏ hidden layers.

```
In [44]: class MNIST_MLR(nn.Module):
          def __init__(self):
              super(MNIST_MLR, self).__init__()
              self.linear = nn.Sequential(nn.Flatten(),
                                          nn.Linear(784, 10))

          def forward(self, x):
              return self.linear(x)

mlr_model = MNIST_MLR()
mlr_module = SimpleModule.classification(mlr_model)
mlr_logger = CSVLogger('logs', name='MNIST_MLR')
```

```
In [45]: mlr_trainer = Trainer(deterministic=True,
                               max_epochs=30,
                               callbacks=[ErrorTracker()])
mlr_trainer.fit(mlr_module, datamodule=mnist_dm)
```

Ta fit model như trước rồi compute test results.

```
In [46]: mlr_trainer.test(mlr_module,
                             datamodule=mnist_dm)
```

```
Out[46]: [{'test_loss': 0.3187, 'test_accuracy': 0.9241}]
```

Accuracy vượt 90% ngay cả với model khá simple này. Như trong `Hitters` example, ta delete một số objects đã create ở trên.

In [47]:

```
del(mnist_test,
    mnist_train,
    mnist_model,
    mnist_dm,
    mnist_trainer,
    mnist_module,
    mnist_results,
    mlr_model,
    mlr_module,
    mlr_trainer)
```

10.9.3 Convolutional Neural Networks

Trong section này ta fit một CNN cho CIFAR100 data, available trong package `torchvision`. Data được arranged tương tự MNIST data.

In [48]:

```
(cifar_train,
cifar_test) = [CIFAR100(root="data",
                        train=train,
                        download=True)
                for train in [True, False]]
```

In [49]:

```
transform = ToTensor()
cifar_train_X = torch.stack([transform(x) for x in
                             cifar_train.data])
cifar_test_X = torch.stack([transform(x) for x in
                             cifar_test.data])
cifar_train = TensorDataset(cifar_train_X,
                             torch.tensor(cifar_train.targets))
cifar_test = TensorDataset(cifar_test_X,
                             torch.tensor(cifar_test.targets))
```

CIFAR100 dataset gồm 50,000 training images, mỗi image represented bởi three-dimensional tensor: mỗi three-color image được represented như set of three channels, mỗi channel gồm 32×32 eight-bit pixels. Ta standardize như đã làm với digits nhưng giữ array structure. Việc này được accomplished bằng transform `ToTensor()`.

Creating data module tương tự MNIST example.

In [50]:

```
cifar_dm = SimpleDataModule(cifar_train,
                            cifar_test,
                            validation=0.2,
                            num_workers=max_num_workers,
                            batch_size=128)
```

Ta lại xem shape của typical batches trong data loaders.

In [51]:

```
for idx, (X_, Y_) in enumerate(cifar_dm.train_dataloader()):
    print('X: ', X_.shape)
    print('Y: ', Y_.shape)
    if idx >= 1:
        break
```

Out[51]:

```
X: torch.Size([128, 3, 32, 32])
Y: torch.Size([128])
X: torch.Size([128, 3, 32, 32])
Y: torch.Size([128])
```

Trước khi bắt đầu, ta xem một số training images; similar code đã produce Figure 10.5 trên page 406. Example dưới cũng illustrate rằng `TensorDataset` objects có thể indexed bằng integers—ta chọn random images từ training data bằng cách indexing `cifar_train`. Để display đúng, ta phải reorder dimensions bằng call tới `np.transpose()`.

In [52]:

```
fig, axes = subplots(5, 5, figsize=(10,10))
rng = np.random.default_rng(4)
indices = rng.choice(np.arange(len(cifar_train)), 25,
                     replace=False).reshape((5,5))

for i in range(5):
    for j in range(5):
        idx = indices[i,j]
        axes[i,j].imshow(np.transpose(cifar_train[idx][0],
                                     [1,2,0]),
                        interpolation=None)
        axes[i,j].set_xticks([])
        axes[i,j].set_yticks([])
```

Ở đây method `imshow()` recognizes từ shape của argument rằng đó là 3-dimensional array, với last dimension indexing three RGB color channels.

`.imshow()`

Ta specify một moderately-sized CNN cho demonstration, similar về structure với Figure 10.8. Ta dùng several layers, mỗi layer gồm convolution, ReLU và max-pooling steps. Trước hết define một module cho một layer như vậy. Như trong previous examples, ta overwrite methods `__init__()` và `forward()` của `nn.Module`. User-defined module này giờ có thể được dùng giống `nn.Linear()` hoặc `nn.Dropout()`.

In [53]:

```
class BuildingBlock(nn.Module):

    def __init__(self,
                 in_channels,
                 out_channels):

        super(BuildingBlock, self).__init__()
        self.conv = nn.Conv2d(in_channels=in_channels,
                              out_channels=out_channels,
                              kernel_size=(3,3),
                              padding='same')
        self.activation = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=(2,2))

    def forward(self, x):
        return self.pool(self.activation(self.conv(x)))
```

Lưu ý argument `padding="same"` cho `nn.Conv2d()` đảm bảo output channels có same dimension như input channels. Có 32 channels trong first hidden layer, so với three channels trong input layer. Ta dùng 3×3 convolution filter cho mỗi channel ở tất cả layers. Mỗi convolution được followed by max-pooling layer trên 2×2 blocks.

Khi forming deep learning model cho CIFAR100 data, ta dùng several `BuildingBlock()` modules sequentially. Simple example này illustrate power của `torch`: users có thể define modules riêng rồi combine chúng trong other modules. Cuối cùng mọi thứ được fit bởi generic trainer.

In [54]:

```
class CIFARModel(nn.Module):

    def __init__(self):
        super(CIFARModel, self).__init__()
        sizes = [(3,32),
                  (32,64),
                  (64,128),
                  (128,256)]
        self.conv = nn.Sequential(*[BuildingBlock(in_, out_)
                                     for in_, out_ in sizes])

        self.output = nn.Sequential(nn.Dropout(0.5),
                                     nn.Linear(2*2*256, 512),
                                     nn.ReLU(),
                                     nn.Linear(512, 100))

    def forward(self, x):
        val = self.conv(x)
        val = torch.flatten(val, start_dim=1)
        return self.output(val)
```

Ta build model và xem summary. (Ta đã create examples của X_1 trước đó.)

In [55]:

```
cifar_model = CIFARModel()
summary(cifar_model,
        input_data=X_,
        col_names=['input_size',
                  'output_size',
                  'num_params'])
```

Out[55]:

```
=====
Layer (type:depth-idx) Input Shape          Output Shape          Param #
=====
CIFARModel              [128, 3, 32, 32]     [128, 100]            --
  Sequential: 1-1       [128, 3, 32, 32]     [128, 256, 2, 2]     --
    BuildingBlock: 2-1  [128, 3, 32, 32]     [128, 32, 16, 16]    --
      Conv2d: 3-1       [128, 3, 32, 32]     [128, 32, 32, 32]    896
      ReLU: 3-2         [128, 32, 32, 32]    [128, 32, 32, 32]    --
      MaxPool2d: 3-3    [128, 32, 32, 32]    [128, 32, 16, 16]    --
    BuildingBlock: 2-2  [128, 32, 16, 16]    [128, 64, 8, 8]      --
      Conv2d: 3-4       [128, 32, 16, 16]    [128, 64, 16, 16]    18,496
      ReLU: 3-5         [128, 64, 16, 16]    [128, 64, 16, 16]    --
      MaxPool2d: 3-6    [128, 64, 16, 16]    [128, 64, 8, 8]      --
    BuildingBlock: 2-3  [128, 64, 8, 8]      [128, 128, 4, 4]     --
      Conv2d: 3-7       [128, 64, 8, 8]      [128, 128, 8, 8]     73,856
      ReLU: 3-8         [128, 128, 8, 8]    [128, 128, 8, 8]     --
      MaxPool2d: 3-9    [128, 128, 8, 8]    [128, 128, 4, 4]     --
    BuildingBlock: 2-4  [128, 128, 4, 4]     [128, 256, 2, 2]     --
      Conv2d: 3-10      [128, 128, 4, 4]     [128, 256, 4, 4]     295,168
      ReLU: 3-11        [128, 256, 4, 4]     [128, 256, 4, 4]     --
      MaxPool2d: 3-12   [128, 256, 4, 4]     [128, 256, 2, 2]     --
  Sequential: 1-2       [128, 1024]           [128, 100]            --
  Dropout: 2-5          [128, 1024]           [128, 1024]           --
  Linear: 2-6           [128, 1024]           [128, 512]           524,800

  ReLU: 2-7             [128, 512]            [128, 512]            --
  Linear: 2-8           [128, 512]            [128, 100]            51,300
=====
Total params: 964,516
Trainable params: 964,516
```

Total number of trainable parameters là 964,516. Nhìn parameter sizes, ta thấy channels halve ở cả two dimensions sau mỗi max-pooling opera-

tion. Sau last one, ta có layer với 256 channels dimension 2×2 . Chúng được flatten thành dense layer size 1,024; tức mỗi matrix 2×2 được turn thành 4-vector rồi placed side-by-side trong một layer. Sau đó là dropout regularization layer, một dense layer khác size 512, và cuối cùng output layer.

Tới đây ta dùng default optimizer trong `SimpleModule()`. Với data này, experiments cho thấy smaller learning rate perform tốt hơn default 0.01. Ta dùng custom optimizer với learning rate 0.001. Ngoài ra logging và training follow pattern tương tự previous examples. Optimizer nhận argument `params` cho biết parameters nào involved trong SGD.

Ta đã thấy entries của module's parameters là tensors. Khi passing parameters tới optimizer, ta làm nhiều hơn việc chỉ pass arrays; một phần structure của graph được encoded trong tensors themselves.

```
In [56]: cifar_optimizer = RMSprop(cifar_model.parameters(), lr=0.001)
cifar_module = SimpleModule.classification(cifar_model,
                                           optimizer=cifar_optimizer)
cifar_logger = CSVLogger('logs', name='CIFAR100')
```

```
In [57]: cifar_trainer = Trainer(deterministic=True,
                                max_epochs=30,
                                logger=cifar_logger,
                                callbacks=[ErrorTracker()])
cifar_trainer.fit(cifar_module,
                  datamodule=cifar_dm)
```

Model này mất 10 minutes hoặc hơn để run và đạt khoảng 42% accuracy trên test data. Dù không quá tệ với 100-class data (random classifier chỉ đạt 1%), searching the web cho thấy results khoảng 75%. Thông thường cần nhiều architecture carpentry, fiddling với regularization và time để đạt results như vậy.

Ta xem validation và training accuracy qua epochs.

```
In [58]: log_path = cifar_logger.experiment.metrics_file_path
cifar_results = pd.read_csv(log_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(cifar_results,
             ax,
             col='accuracy',
             ylabel='Accuracy')
ax.set_xticks(np.linspace(0, 10, 6).astype(int))
ax.set_ylabel('Accuracy')
ax.set_ylim([0, 1]);
```

Cuối cùng evaluate model trên test data.

```
In [59]: cifar_trainer.test(cifar_module,
                           datamodule=cifar_dm)
```

```
Out[59]: [{'test_loss': 2.4238 'test_accuracy': 0.4206}]
```

Hardware Acceleration

Khi deep learning trở nên ubiquitous trong machine learning, hardware manufacturers đã produce special libraries có thể speed up gradient-descent

steps.

Chẳng hạn Mac OS devices với M1 chip có thể enable Metal programming framework, giúp speed up torch computations. Ta trình bày example cách dùng acceleration này.

Main changes nằm ở call `Trainer()` cũng như metrics sẽ được evaluated trên data. Các metrics phải được told nơi data sẽ located ở evaluation time. Điều này accomplished bằng call method `to()` của metrics.

In [60]:

```
try:
    for name, metric in cifar_module.metrics.items():
        cifar_module.metrics[name] = metric.to('mps')
    cifar_trainer_mps = Trainer(accelerator='mps',
                               deterministic=True,
                               max_epochs=30)
    cifar_trainer_mps.fit(cifar_module,
                          datamodule=cifar_dm)
    cifar_trainer_mps.test(cifar_module,
                           datamodule=cifar_dm)
except:
    pass
```

Cách này yield approximately two- or three-fold acceleration cho mỗi epoch. Code block được protect bằng clauses `try:` và `except:`; nếu works thì ta có speedup, nếu fails thì không có gì xảy ra.

10.9.4 Using Pretrained CNN Models

Bây giờ ta cho thấy cách dùng CNN pretrained trên imagenet database để classify natural images, đồng thời demonstrate cách produce Figure 10.10. Ta copy six JPEG images từ digital photo album vào directory `book_images`. Các images available từ data section của www.statlearning.com, ISLP book website. Download `book_images.zip`; khi click, nó create directory `book_images`.

Pretrained network ta dùng gọi là `resnet50`; specification details có thể found on the web. Ta read images và convert sang array format expected bởi torch software để match specifications trong `resnet50`. Conversion gồm resize, crop, rồi predefined standardization cho mỗi of three channels. Ta read và preprocess images.

In [61]:

```
resize = Resize((232,232))
crop = CenterCrop(224)
normalize = Normalize([0.485,0.456,0.406],
                      [0.229,0.224,0.225])
imgfiles = sorted([f for f in glob('book_images/*')])
imgs = torch.stack([torch.div(crop(resize(read_image(f))), 255)
                    for f in imgfiles])
imgs = normalize(imgs)
imgs.size()
```

Out[61]:

```
torch.Size([6, 3, 224, 224])
```

Ta setup trained network với weights đã read trong code block 6. Model có 50 layers, với khá nhiều complexity.

In [62]:

```
resnet_model = resnet50(weights=ResNet50_Weights.DEFAULT)
summary(resnet_model,
```

```
input_data=imgs,
col_names=['input_size',
           'output_size',
           'num_params'])
```

Ta set mode thành `eval()` để đảm bảo model ready để predict trên new data.

```
In [63]: resnet_model.eval()
```

Inspecting output ở trên, ta thấy khi setup `resnet_model`, authors define một Bottleneck, tương tự module `BuildingBlock` của ta. Bây giờ feed six images qua fitted network.

```
In [64]: img_preds = resnet_model(imgs)
```

Ta xem predicted probabilities cho top 3 choices của mỗi image. Trước hết compute probabilities bằng cách apply softmax lên logits trong `img_preds`. Lưu ý phải call method `detach()` trên tensor `img_preds` để convert nó thành familiar `ndarray`.

```
In [65]: img_probs = np.exp(np.asarray(img_preds.detach()))
img_probs /= img_probs.sum(1)[:,None]
```

Để thấy class labels, ta phải download index file associated với imagenet.²⁷

```
In [66]: labs = json.load(open('imagenet_class_index.json'))
class_labels = pd.DataFrame([(int(k), v[1]) for k, v in
                             labs.items()],
                             columns=['idx', 'label'])
class_labels = class_labels.set_index('idx')
class_labels = class_labels.sort_index()
```

Bây giờ construct data frame cho mỗi image file với labels có three highest probabilities estimated bởi model.

```
In [67]: for i, imgfile in enumerate(imgfiles):
img_df = class_labels.copy()
img_df['prob'] = img_probs[i]
img_df = img_df.sort_values(by='prob', ascending=False)[:3]
print(f'Image: {imgfile}')
print(img_df.reset_index().drop(columns=['idx']))
```

```
Out[67]: Image: book_images/Cape_Weaver.jpg
          label      prob
0   jacamar  0.287283
1  bee_eater  0.046768
2   bulbul   0.037507
Image: book_images/Flamingo.jpg
          label      prob
0   flamingo  0.591761
1  spoonbill  0.012386
2  American_egret  0.002105
Image: book_images/Hawk_Fountain.jpg
          label      prob
```

²⁷File này available từ book website và https://s3.amazonaws.com/deep-learning-models/image-models/imagenet_class_index.json.

```

0 great_grey_owl 0.287959
1 kite 0.039478
2 fountain 0.029384
Image: book_images/Hawk_cropped.jpg
  label      prob
0 kite 0.301830
1 jay 0.121674
2 magpie 0.015513
Image: book_images/Lhasa_Apso.jpg
  label      prob
0 Lhasa 0.151143
1 Shih-Tzu 0.129850
2 Tibetan_terrier 0.102358
Image: book_images/Sleeping_Cat.jpg
  label      prob
0 tabby 0.173627
1 tiger_cat 0.110414
2 doormat 0.093447

```

Ta thấy model khá confident với `Flamingo.jpg`, nhưng ít hơn với các images còn lại. Ta kết thúc section bằng usual cleanup.

In [68]:

```

del(cifar_test,
    cifar_train,
    cifar_dm,
    cifar_module,
    cifar_logger,
    cifar_optimizer,
    cifar_trainer)

```

10.9.5 IMDB Document Classification

Bây giờ ta implement models cho sentiment classification (Section 10.4) trên IMDB dataset. Như đã mentioned trước code block 8, ta dùng pre-processed version của IMDB dataset từ package `keras`. Vì `keras` dùng `tensorflow`, một tensor và deep learning library khác, ta đã convert data để suitable cho `torch`. Code dùng để convert từ `keras` available trong module `ISLP.torch._make_imdb`. Nó requires một số `keras` packages để run. Data này dùng dictionary size 10,000.

Ta stored three different representations của review data cho lab này:

- `load_tensor()`, sparse tensor version usable bởi `torch`;
- `load_sparse()`, sparse matrix version usable bởi `sklearn`, vì ta sẽ compare với lasso fit;
- `load_sequential()`, padded version của original sequence representation, limited tới last 500 words của mỗi review.

In [69]:

```

(imdb_seq_train,
 imdb_seq_test) = load_sequential(root='data/IMDB')
padded_sample = np.asarray(imdb_seq_train.tensors[0][0])
sample_review = padded_sample[padded_sample > 0][:12]
sample_review[:12]

```

Out[69]:

```

array([ 1, 14, 22, 16, 43, 530, 973, 1622, 1385,
        65, 458, 4468], dtype=int32)

```

Datasets `imdb_seq_train` và `imdb_seq_test` đều là instances của class

TensorDataset. Tensors dùng để construct chúng nằm trong attribute **tensors**, với first tensor là features X và second là outcome Y . Ta lấy first row của features và store thành **padded_sample**. Trong preprocessing dùng để form data, sequences được padded bằng 0s ở beginning nếu không đủ dài; vì vậy remove padding bằng cách restrict tới entries **padded_sample** > 0. Sau đó provide first 12 words của sample review.

Ta tìm các words này trong lookup dictionary từ module **ISLP.torch.imdb**.

```
In [70]: lookup = load_lookup(root='data/IMDB')
         ' '.join(lookup[i] for i in sample_review)
```

```
Out[70]: "<START> this film was just brilliant casting location scenery
         story direction everyone's"
```

Với first model, ta create binary feature cho mỗi of 10,000 possible words trong dataset, với entry 1 ở vị trí i, j nếu word j appears trong review i . Vì most reviews khá short, feature matrix như vậy có hơn 98% zeros. Data được access bằng **load_tensor()** từ **ISLP** library.

```
In [71]: max_num_workers=10
         (imdb_train,
          imdb_test) = load_tensor(root='data/IMDB')
         imdb_dm = SimpleDataModule(imdb_train,
                                   imdb_test,
                                   validation=2000,
                                   num_workers=min(6, max_num_workers),
                                   batch_size=512)
```

Ta dùng two-layer model cho first model.

```
In [72]: class IMDBModel(nn.Module):

         def __init__(self, input_size):
             super(IMDBModel, self).__init__()
             self.dense1 = nn.Linear(input_size, 16)
             self.activation = nn.ReLU()
             self.dense2 = nn.Linear(16, 16)
             self.output = nn.Linear(16, 1)

         def forward(self, x):
             val = x
             for _map in [self.dense1,
                           self.activation,
                           self.dense2,
                           self.activation,
                           self.output]:
                 val = _map(val)
             return torch.flatten(val)
```

Ta instantiate model và xem summary (không shown).

```
In [73]: imdb_model = IMDBModel(imdb_test.tensors[0].size()[1])
         summary(imdb_model,
                 input_size=imdb_test.tensors[0].size(),
                 col_names=['input_size',
                           'output_size',
                           'num_params'])
```

Ta lại dùng smaller learning rate cho data này, nên pass optimizer tới

`SimpleModule`. Vì reviews được classified thành positive hoặc negative sentiment, ta dùng `SimpleModule.binary_classification()`.²⁸

```
In [74]: imdb_optimizer = RMSprop(imdb_model.parameters(), lr=0.001)
imdb_module = SimpleModule.binary_classification(
    imdb_model,
    optimizer=imdb_optimizer)
```

Sau khi load datasets vào data module và create `SimpleModule`, remaining steps đã familiar.

```
In [75]: imdb_logger = CSVLogger('logs', name='IMDB')
imdb_trainer = Trainer(deterministic=True,
    max_epochs=30,
    logger=imdb_logger,
    callbacks=[ErrorTracker()])
imdb_trainer.fit(imdb_module,
    datamodule=imdb_dm)
```

Evaluating test error yield roughly 86% accuracy.

```
In [76]: test_results = imdb_trainer.test(imdb_module, datamodule=imdb_dm)
test_results
```

```
Out[76]: [{'test_loss': 1.0863, 'test_accuracy': 0.8550}]
```

Comparison to Lasso

Ta fit lasso logistic regression model bằng `LogisticRegression()` từ `sklearn`. Vì `sklearn` không recognize sparse tensors của `torch`, ta dùng sparse matrix mà `sklearn` recognize.

```
In [77]: ((X_train, Y_train),
    (X_valid, Y_valid),
    (X_test, Y_test)) = load_sparse(validation=2000,
    random_state=0,
    root='data/IMDB')
```

Tương tự Section 10.9.1, ta construct series 50 values cho lasso regularization parameter λ .

```
In [78]: lam_max = np.abs(X_train.T * (Y_train - Y_train.mean())).max()
lam_val = lam_max * np.exp(np.linspace(np.log(1),
    np.log(1e-4), 50))
```

Với `LogisticRegression()`, regularization parameter C được specified như inverse của λ . Có several solvers cho logistic regression; ở đây dùng `liblinear`, hoạt động tốt với sparse input format.

```
In [79]: logit = LogisticRegression(penalty='l1',
    C=1/lam_max,
    solver='liblinear',
    warm_start=True,
    fit_intercept=True)
```

²⁸Việc dùng `binary_classification()` thay vì `classification()` xuất phát từ một subtlety trong cách `torchmetrics.Accuracy()` hoạt động, cũng như data type của targets.

Path gồm 50 values mất khoảng 40 seconds để run.

In [80]:

```
coefs = []
intercepts = []

for l in lam_val:
    logit.C = 1/l
    logit.fit(X_train, Y_train)
    coefs.append(logit.coef_.copy())
    intercepts.append(logit.intercept_)
```

Coefficients và intercepts có một extraneous dimension có thể removed bằng function `np.squeeze()`.

In [81]:

```
coefs = np.squeeze(coefs)
intercepts = np.squeeze(intercepts)
```

Ta tạo plot để compare neural network results với lasso.

In [82]:

```
%%capture
fig, axes = subplots(1, 2, figsize=(16, 8), sharey=True)
for ((X_, Y_),
     data_,
     color) in zip([(X_train, Y_train),
                    (X_valid, Y_valid),
                    (X_test, Y_test)],
                  ['Training', 'Validation', 'Test'],
                  ['black', 'red', 'blue']):
    linpred_ = X_ * coefs.T + intercepts[None,:]
    label_ = np.array(linpred_ > 0)
    accuracy_ = np.array([np.mean(Y_ == 1) for l in label_.T])
    axes[0].plot(-np.log(lam_val / X_train.shape[0]),
                 accuracy_,
                 '--',
                 color=color,
                 markersize=13,
                 linewidth=2,
                 label=data_)

axes[0].legend()
axes[0].set_xlabel(r'$-\log(\lambda)$', fontsize=20)
axes[0].set_ylabel('Accuracy', fontsize=20)
```

Lưu ý use của `%%capture`, suppress display của partially completed figure. Điều này useful khi making complex figure vì steps có thể spread qua two hoặc more cells. Ta add plot của lasso accuracy và display composed figure bằng cách simply entering name ở end of cell.

%%capture

In [83]:

```
imdb_results = pd.read_csv(imdb_logger.experiment.metrics_file_path)
summary_plot(imdb_results,
             axes[1],
             col='accuracy',
             ylabel='Accuracy')
axes[1].set_xticks(np.linspace(0, 30, 7).astype(int))
axes[1].set_ylabel('Accuracy', fontsize=20)
axes[1].set_xlabel('Epoch', fontsize=20)
axes[1].set_ylim([0.5, 1]);
axes[1].axhline(test_results[0]['test_accuracy'],
                color='blue',
                linestyle='--',
                linewidth=3)

fig
```

Từ graphs ta thấy accuracy của lasso logistic regression peaks ở khoảng 0.88, tương tự neural network. Một lần nữa, kết thúc bằng cleanup.

In [84]:

```
del(imdb_model,
    imdb_trainer,
    imdb_logger,
    imdb_dm,
    imdb_train,
    imdb_test)
```

10.9.6 Recurrent Neural Networks

Trong lab này ta fit các models illustrated trong Section 10.5.

Sequential Models for Document Classification

Ở đây ta fit một simple LSTM RNN cho sentiment prediction trên IMDB movie-review data như discussed trong Section 10.5.1. Với RNN, ta dùng sequence of words trong document, taking order into account. Ta đã load preprocessed data ở beginning của Section 10.9.5. Một script detailing preprocessing có trong ISLP library. Đáng chú ý, vì hơn 90% documents có fewer than 500 words, ta set document length bằng 500. Với longer documents, dùng last 500 words; với shorter documents, pad front bằng blanks.

In [85]:

```
imdb_seq_dm = SimpleDataModule(imdb_seq_train,
                               imdb_seq_test,
                               validation=2000,
                               batch_size=300,
                               num_workers=min(6, max_num_workers)
                               )
```

First layer của RNN là embedding layer size 32, sẽ được learned trong training. Layer này one-hot encode mỗi document thành matrix dimension $500 \times 10,003$, rồi map 10,003 dimensions này xuống 32.²⁹ Vì mỗi word represented bởi integer, điều này effectively achieved bằng creation of embedding matrix size $10,003 \times 32$; mỗi of 500 integers trong document sau đó mapped tới appropriate 32 real numbers bằng indexing appropriate rows của matrix.

Second layer là LSTM với 32 units, còn output layer là single logit cho binary classification task. Trong last line của method `forward()` dưới đây, ta lấy last 32-dimensional output của LSTM rồi map nó tới response.

In [86]:

```
class LSTMModel(nn.Module):
    def __init__(self, input_size):
        super(LSTMModel, self).__init__()
        self.embedding = nn.Embedding(input_size, 32)
        self.lstm = nn.LSTM(input_size=32,
                            hidden_size=32,
                            batch_first=True)
        self.dense = nn.Linear(32, 1)
    def forward(self, x):
        val, (h_n, c_n) = self.lstm(self.embedding(x))
        return torch.flatten(self.dense(val[:,-1]))
```

Ta instantiate và xem summary của model, dùng first 10 documents trong

²⁹Extra 3 dimensions correspond tới commonly occurring non-word entries trong reviews.

corpus.

```
In [87]: lstm_model = LSTMModel(X_test.shape[-1])
summary(lstm_model,
        input_data=imdb_seq_train.tensors[0][:10],
        col_names=['input_size',
                    'output_size',
                    'num_params'])
```

```
Out[87]: =====
Layer (type:depth-idx)   Input Shape   Output Shape   Param #
=====
LSTMModel                [10, 500]     [10]           --

Out[87]: Embedding: 1-1      [10, 500]     [10, 500, 32]   320,096
          LSTM: 1-2         [10, 500, 32] [10, 500, 32]   8,448
          Linear: 1-3       [10, 32]      [10, 1]         33
          =====
Total params: 328,577
Trainable params: 328,577
```

Value 10,003 bị suppressed trong summary, nhưng ta thấy nó qua parameter count vì $10,003 \times 32 = 320,096$.

```
In [88]: lstm_module = SimpleModule.binary_classification(lstm_model)
lstm_logger = CSVLogger('logs', name='IMDB_LSTM')
```

```
In [89]: lstm_trainer = Trainer(deterministic=True,
                                max_epochs=20,
                                logger=lstm_logger,
                                callbacks=[ErrorTracker()])
lstm_trainer.fit(lstm_module,
                 datamodule=imdb_seq_dm)
```

Phần còn lại tương tự other networks đã fit. Ta track test performance trong quá trình fitting và thấy model attain 85% accuracy.

```
In [90]: lstm_trainer.test(lstm_module, datamodule=imdb_seq_dm)
```

```
Out[90]: [{'test_loss': 0.8178, 'test_accuracy': 0.8476}]
```

Ta một lần nữa show learning progress, followed by cleanup.

```
In [91]: lstm_results = pd.read_csv(lstm_logger.experiment.metrics_file_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(lstm_results,
             ax,
             col='accuracy',
             ylabel='Accuracy')
ax.set_xticks(np.linspace(0, 20, 5).astype(int))
ax.set_ylabel('Accuracy')
ax.set_ylim([0.5, 1])
```

```
In [92]: del(lstm_model,
            lstm_trainer,
            lstm_logger,
            imdb_seq_dm,
```

```
imdb_seq_train,
imdb_seq_test)
```

Time Series Prediction

Ta show cách fit models trong Section 10.5.2 cho time series prediction. Trước hết load và standardize data.

```
In [93]: NYSE = load_data('NYSE')
cols = ['DJ_return', 'log_volume', 'log_volatility']
X = pd.DataFrame(StandardScaler(
    with_mean=True,
    with_std=True).fit_transform(NYSE[cols]),
    columns=NYSE[cols].columns,
    index=NYSE.index)
```

Tiếp theo setup lagged versions của data, dropping rows có missing values bằng method `dropna()`.

```
In [94]: for lag in range(1, 6):
    for col in cols:
        newcol = np.zeros(X.shape[0]) * np.nan
        newcol[lag:] = X[col].values[:-lag]
        X.insert(len(X.columns), "{0}_{1}".format(col, lag), newcol)
X.insert(len(X.columns), 'train', NYSE['train'])
X = X.dropna()
```

Cuối cùng extract response, training indicator, và drop current day's DJ_return và log_volatility để predict chỉ từ previous day's data.

```
In [95]: Y, train = X['log_volume'], X['train']
X = X.drop(columns=['train'] + cols)
X.columns
```

```
Out[95]: Index(['DJ_return_1', 'log_volume_1', 'log_volatility_1',
    'DJ_return_2', 'log_volume_2', 'log_volatility_2',
    'DJ_return_3', 'log_volume_3', 'log_volatility_3',
    'DJ_return_4', 'log_volume_4', 'log_volatility_4',
    'DJ_return_5', 'log_volume_5', 'log_volatility_5'],
    dtype='object')
```

Trước hết fit simple linear model và compute R^2 trên test data bằng method `score()`.

```
In [96]: M = LinearRegression()
M.fit(X[train], Y[train])
M.score(X[-train], Y[-train])
```

```
Out[96]: 0.4129
```

Ta refit model này, including factor variable `day_of_week`. Với categorical series trong pandas, ta có thể form indicators bằng method `get_dummies()`.

```
In [97]: X_day = pd.merge(X,
    pd.get_dummies(NYSE['day_of_week']),
    on='date')
```

Lưu ý không cần reinstantiate linear regression model vì method `fit()` accept design matrix và response trực tiếp.

```
In [98]: M.fit(X_day[train], Y[train])
         M.score(X_day[-train], Y[-train])
```

```
Out[98]: 0.4595
```

Model này đạt R^2 khoảng 46%.

Để fit RNN, ta phải reshape data vì nó expect 5 lagged versions của mỗi feature như indicated bởi `input_shape` argument tới layer `nn.RNN()` bên dưới. Trước hết ensure columns của data frame sao cho reshaped matrix có variables lagged đúng. Ta dùng method `reindex()`.

Với input shape (5,3), mỗi row represent một lagged version của three variables. Layer `nn.RNN()` cũng expect first row của mỗi observation là earliest in time, nên phải reverse current order. Vì vậy ta loop over `range(5,0,-1)`, một example dùng `slice()` để index iterable objects. General notation là `start:end:step`.

```
In [99]: ordered_cols = []
         for lag in range(5,0,-1):
             for col in cols:
                 ordered_cols.append('{0}_{1}'.format(col, lag))
         X = X.reindex(columns=ordered_cols)
         X.columns
```

```
Out[99]: Index(['DJ_return_5', 'log_volume_5', 'log_volatility_5',
              'DJ_return_4', 'log_volume_4', 'log_volatility_4',
              'DJ_return_3', 'log_volume_3', 'log_volatility_3',
              'DJ_return_2', 'log_volume_2', 'log_volatility_2',
              'DJ_return_1', 'log_volume_1', 'log_volatility_1'],
              dtype='object')
```

Bây giờ reshape data.

```
In [100]: X_rnn = X.to_numpy().reshape((-1,5,3))
          X_rnn.shape
```

```
Out[100]: (6046, 5, 3)
```

Khi specify first size bằng -1, `numpy.reshape()` deduces size dựa trên remaining arguments.

Bây giờ proceed với RNN dùng 12 hidden units và 10% dropout. Sau khi pass qua RNN, ta extract final time point như `val[:,-1]` trong `forward()` bên dưới. Nó được pass qua 10% dropout rồi flattened qua linear layer.

```
In [101]: class NYSEModel(nn.Module):
         def __init__(self):
             super(NYSEModel, self).__init__()
             self.rnn = nn.RNN(3,
                               12,
                               batch_first=True)
             self.dense = nn.Linear(12, 1)
             self.dropout = nn.Dropout(0.1)
         def forward(self, x):
             val, h_n = self.rnn(x)
```

```

        val = self.dense(self.dropout(val[:,-1]))
        return torch.flatten(val)
nyse_model = NYSEModel()

```

Ta fit model tương tự previous networks. Ta supply fit function với test data như validation data, để khi monitor progress và plot history function có thể thấy progress trên test data. Dĩ nhiên không nên dùng điều này làm basis cho early stopping vì khi đó test performance sẽ biased. Ta form training dataset tương tự Hitters example.

```

In [102]: datasets = []
for mask in [train, ~train]:
    X_rnn_t = torch.tensor(X_rnn[mask].astype(np.float32))
    Y_t = torch.tensor(Y[mask].astype(np.float32))
    datasets.append(TensorDataset(X_rnn_t, Y_t))
nyse_train, nyse_test = datasets

```

Theo usual pattern, ta inspect summary.

```

In [103]: summary(nyse_model,
                  input_data=X_rnn_t,
                  col_names=['input_size',
                             'output_size',
                             'num_params'])

```

```

Out[103]: =====
Layer (type:depth-idx)   Input Shape      Output Shape     Param #
=====
NYSEModel                [1770, 5, 3]    [1770]           --
  RNN: 1-1                [1770, 5, 3]    [1770, 5, 12]    204
  Dropout: 1-2            [1770, 12]       [1770, 12]       --
  Linear: 1-3              [1770, 12]       [1770, 1]        13
=====
Total params: 217
Trainable params: 217

```

Ta lại put two datasets vào data module, với batch size 64.

```

In [104]: nyse_dm = SimpleDataModule(nyse_train,
                                     nyse_test,
                                     num_workers=min(4, max_num_workers),
                                     validation=nyse_test,
                                     batch_size=64)

```

Ta run some data qua model để chắc chắn sizes match đúng.

```

In [105]: for idx, (x, y) in enumerate(nyse_dm.train_dataloader()):
            out = nyse_model(x)
            print(y.size(), out.size())
            if idx >= 2:
                break

```

```

Out[105]: torch.Size([64]) torch.Size([64])
torch.Size([64]) torch.Size([64])
torch.Size([64]) torch.Size([64])

```

Ta follow previous example để setup trainer cho regression problem, requesting R^2 metric được compute ở mỗi epoch.

```
In [106]: nyse_optimizer = RMSprop(nyse_model.parameters(),
                                   lr=0.001)
nyse_module = SimpleModule.regression(nyse_model,
                                       optimizer=nyse_optimizer,
                                       metrics={'r2': R2Score()})
```

Fitting model giờ đã familiar. Results trên test data rất similar với linear AR model.

```
In [107]: nyse_trainer = Trainer(deterministic=True,
                                max_epochs=200,
                                callbacks=[ErrorTracker()])
nyse_trainer.fit(nyse_module,
                 datamodule=nyse_dm)
nyse_trainer.test(nyse_module,
                  datamodule=nyse_dm)
```

```
Out[107]: [{'test_loss': 0.6141, 'test_r2': 0.4172}]
```

Ta cũng có thể fit model without layer `nn.RNN()` bằng cách chỉ dùng `nn.Flatten()` layer. Đây sẽ là nonlinear AR model. Nếu ngoài ra exclude hidden layer, nó sẽ equivalent với earlier linear AR model.

Thay vào đó ta fit nonlinear AR model bằng feature set `X_day`, gồm các `day_of_week` indicators. Để làm vậy, trước hết phải create test và training datasets và corresponding data module. Việc này có vẻ hơi burdensome nhưng là part of general pipeline cho `torch`.

```
In [108]: datasets = []
for mask in [train, ~train]:
    X_day_t = torch.tensor(
        np.asarray(X_day[mask]).astype(np.float32))
    Y_t = torch.tensor(np.asarray(Y[mask]).astype(np.float32))
    datasets.append(TensorDataset(X_day_t, Y_t))
day_train, day_test = datasets
```

Creating data module follows familiar pattern.

```
In [109]: day_dm = SimpleDataModule(day_train,
                                    day_test,
                                    num_workers=min(4, max_num_workers),
                                    validation=day_test,
                                    batch_size=64)
```

Ta build `NonLinearARModel()` nhận 20 features làm input và hidden layer 32 units. Remaining steps familiar.

```
In [110]: class NonLinearARModel(nn.Module):
    def __init__(self):
        super(NonLinearARModel, self).__init__()
        self._forward = nn.Sequential(nn.Flatten(),
                                       nn.Linear(20, 32),
                                       nn.ReLU(),
                                       nn.Dropout(0.5),
                                       nn.Linear(32, 1))

    def forward(self, x):
        return torch.flatten(self._forward(x))
```

In $[111]$:

[illegible]